

Reviewer Pack — Minimal Rank of Noncommutative Crossed Products over Functio...

Entry cf197e365a12 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 19:12:22 UTC

Minimal Rank of Noncommutative Crossed Products over Function Fields vs Resolution Proof Width

Entry ID: cf197e365a12 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 19:12:22 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Crossed Products) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every Boolean function f with n variables, let G be the noncommutative crossed product of the group ring $\mathbb{Z}[2^n]$ with itself by the antipodal map that interchanges $+1$ and -1 elements. The minimal rank of the module structure on G induced by the Boolean function f is bounded above by the resolution proof width for the corresponding CNF encoding of f .’, ‘Specifically, let $r(G_f)$ be the minimal number of generators needed to represent the module structure over G induced by f . Then there exists a constant c such that $r(G_f) \leq c * w(\text{CNF}(f))$ ’, ‘where $w(\text{CNF}(f))$ is the resolution proof width for the CNF encoding of f .’]

Rationale (proposer’s reasoning):

[‘Noncommutative geometry, particularly crossed products, has been used to study various aspects of quantum systems and algebraic structures. This conjecture suggests that the minimal rank of noncommutative crossed products could be a useful tool in complexity theory, providing insights into the structure of resolution proofs.’, ‘The connection between noncommutative crossed products and Boolean functions is relatively unexplored. This conjecture would bridge this gap and potentially lead to new algorithms or lower bounds for resolution proof complexity.’]

Taxonomy category: NONCOMMUTATIVE_GEOMETRY_X_RESOLUTION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 48452cfe7067d375

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across at least 24 out of 30 random seeds, the ratio $r(G_f) / w(\text{CNF}(f))$ does not exceed a constant c for all $n \leq 40$ Boolean functions f .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncommutative geometry crossed products' AND 'resolution proof complexity' - 'minimal rank noncommutative crossed products' AND 'CNF resolution proof width' - 'group ring $Z[2^n]$ antipodal map' AND 'resolution proof complexity'

Top relevant hits considered: - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Non-commutative Algebraic Geometry and Noncommutative Tori - [http://arxiv.org/abs/1304.4062v2] Superconformal nets and noncommutative geometry - [http://arxiv.org/abs/1209.3595v2] Non-commutative complex differential geometry - [http://arxiv.org/abs/0705.1265v2] A noncommutative Bohnenblust-Spitzer identity for Rota-Baxter algebras solves Bogoliubov's recursion - [http://arxiv.org/abs/1411.6751v4] The noncommutative infinitesimal equivariant index formula: part II - [http://arxiv.org/abs/2005.06373v2] Counting Schur Rings over Cyclic Groups of Semi-prime Order - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order $p^a q$ - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p -groups of rank 2

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.8s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_cnf(n):
    cnf = []
    for i in range(1 << n):
        clause = [random.choice([-1, 1]) * (j + 1) for j in range(n)]
        if all(clause[j] != -clause[(j + 1) % n] for j in range(n)):
            cnf.append(clause)
    return cnf

def resolution_width(cnf):
    clauses = set(tuple(sorted(clause)) for clause in cnf)
    queue = list(clauses)
    while queue:
        clause1, clause2 = queue.pop()
        for lit in clause1:
            if -lit in clause2:
                new_clause = tuple(sorted(set(clause1) | set(clause2) - {lit, -lit}))
```

```

        if len(new_clause) == 1:
            return len(queue)
        if new_clause not in clauses:
            queue.append(new_clause)
            clauses.add(new_clause)
    return float('inf')

def noncommutative_crossed_product(cnf):
    n = int(cnf[0][0].bit_length() - 1)
    G_f = [[0] * (2 ** (n + 1)) for _ in range(2 ** (n + 1))]
    for clause in cnf:
        for lit in clause:
            i, j = divmod(abs(lit) - 1, n)
            sign = -1 if lit < 0 else 1
            for k in range(n):
                G_f[i * (1 << n) + j][k * (1 << n) + (j ^ lit)] += sign
    return G_f

def min_rank(G_f):
    rows, cols = len(G_f), len(G_f[0])
    rank = 0
    for i in range(rows):
        if any(G_f[i][j] != 0 for j in range(cols)):
            rank += 1
            for j in range(cols):
                if G_f[i][j] != 0:
                    for k in range(rows):
                        G_f[k][j] -= (G_f[k][i] * G_f[i][j]) / G_f[i][i]
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    r_G_f = min_rank(noncommutative_crossed_product(cnf))
    w_CNF_f = resolution_width(cnf)
    if w_CNF_f == float('inf'):
        return {
            "metric_name": "min_rank_over_resolution_width",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "resolution_width_infinite"
        }
    c = 2 # Example constant, adjust as needed
    conjecture_holds = r_G_f <= c * w_CNF_f
    return {
        "metric_name": "min_rank_over_resolution_width",
        "metric_value": r_G_f / w_CNF_f,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"c={c}, r(G_f)={r_G_f}, w(CNF(f))={w_CNF_f}"
    }

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {{'seed': {seed}, **trial_result}}")
    results.append(trial_result)

if all("conjecture_holds" not in result or result["conjecture_holds"] for result in results):
    support_fraction = sum(1 for result in results if "conjecture_holds" in result and result["conjecture_holds"])
    RESULT = f"SUPPORTED mean={sum(result['metric_value'] for result in results) / len(results)}"
elif any("counterexample" in result for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if "conjecture_holds" not in result)
    RESULT = f"FALSIFIED counterexample=\"{next(result['counterexample'] for result in results)}\"
else:
    RESULT = "INCONCLUSIVE no_data"

print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Retry the experiment with a longer time limit or increase the number of seeds to ensure sufficient data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13027
2	preregistration	ollama_remote	glm4:latest	0	0	5827
3	novelty	ollama_remote	glm4:latest	0	0	4761
4	novelty	ollama_remote	glm4:latest	0	0	6250
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18097
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12471
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10972
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12231

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	41650

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125285 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/cf197e365a12.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cf197e365a12.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cf197e365a12.tar.gz (if generated)